

Lab 07 – Worksheet

Name: Arsheel Abbas, Emaan Fatima, Abdul Ghani	ID:11163, 11031,11943	Section: T3
--	-----------------------	-------------

b. Selective 2's complement

Inputs to complement				Outputs of the complement		
O	B ₂	B ₁	B ₀	C ₂	C ₁	C ₀
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	1	0
0	0	1	1	0	1	1
0	1	0	0	1	0	0
0	1	0	1	1	0	1
0	1	1	0	1	1	0
0	1	1	1	1	1	1
1	0	0	0	0	0	0
1	0	0	1	1	1	1
1	0	1	0	1	1	0
1	0	1	1	1	0	1
1	1	0	0	1	0	0
1	1	0	1	0	1	1
1	1	1	0	0	1	0

1	1	1	1	0	0	1
---	---	---	---	---	---	---

Write expressions for C_2 , C_1 and C_0

Table 7.2

$C_2 =$	$O'B_2 + B_2B_1'B_0' + OB_2'B_0 + O B_2'B_1$
$C_1 =$	$O'B_1 + B_1B_0 + OB_1B_0$
$C_0 =$	B_0

Provide appropriately commented code for your selective 2's complement module

```
module complement (
    input O,
    input [2:0] B,
    output [2:0] C
);

    // Logic for C0
    assign C[0] = B[0];

    // Logic for C1
    assign C[1] = (~O & B[1]) | (B[1] & ~B[0]) | (O & ~B[1] & B[0]);

    // Logic for C2
    assign C[2] = (~O & B[2]) | (B[2] & ~B[1] & ~B[0]) | (O & ~B[2] & B[0]) | (O & ~B[2]
    & B[1]);

endmodule
```

Write a testbench to thoroughly test designed complement module.

```
`timescale 1ns / 1ps

module tb_complement();

    // Inputs to the module (declared as reg)
    reg O;
    reg [2:0] B;

    // Outputs from the module (declared as wire)
    wire [2:0] C;

    // Instantiate the Unit Under Test (UUT)
    complement uut (
        .O(O),
        .B(B),
        .C(C)
    );

    initial begin
```

```
// Monitor changes in the console for your log window requirement
$monitor("Time=%0d | O=%b | B=%b | Output C=%b", $time, O, B, C);

// --- Mode: Pass Through (O = 0) ---
O = 0;

B = 3'b001; #10; // Case 1: Input 1, Output should be 1
B = 3'b100; #10; // Case 2: Input 4, Output should be 4
B = 3'b111; #10; // Case 3: Input 7, Output should be 7

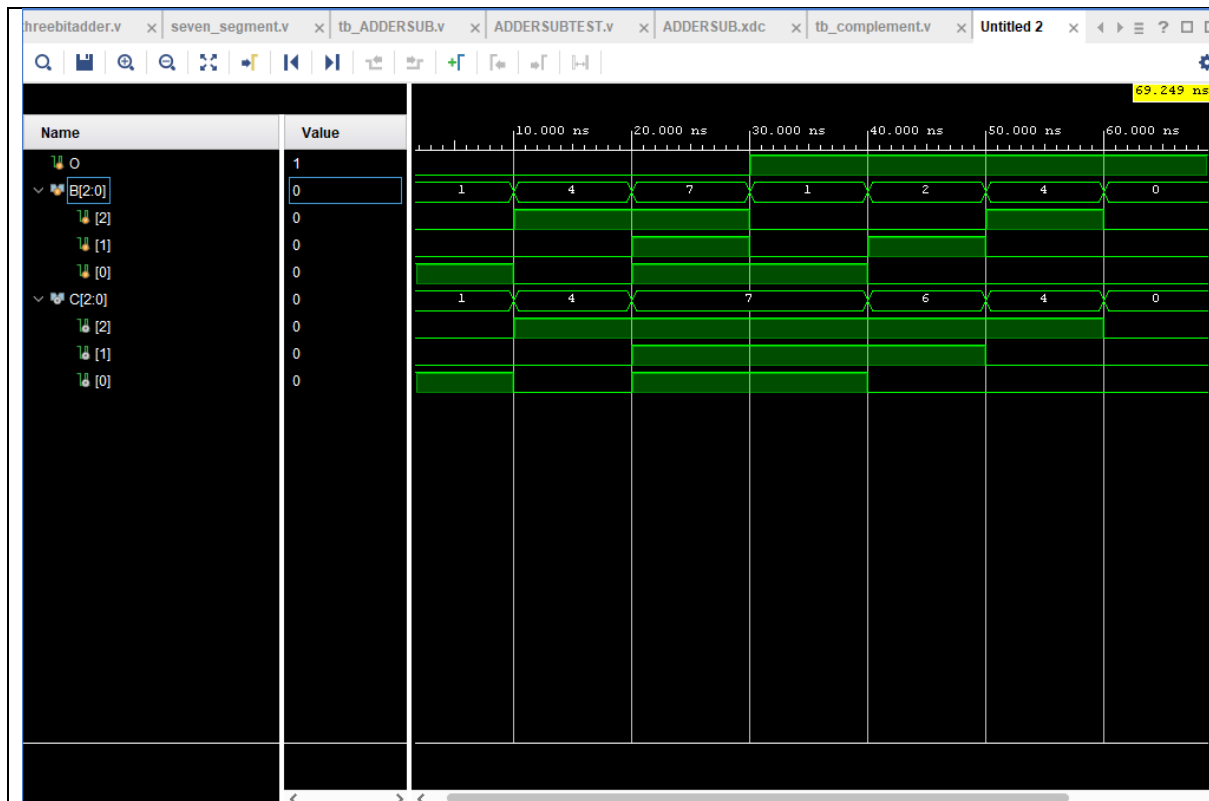
// --- Mode: 2's Complement (O = 1) ---
O = 1;

B = 3'b001; #10; // Case 4: 2's complement of 1 is 111 (7)
B = 3'b010; #10; // Case 5: 2's complement of 2 is 110 (6)
B = 3'b100; #10; // Case 6: 2's complement of 4 is 100 (4)
B = 3'b000; #10; // Case 7: 2's complement of 0 is 000 (0)

$finish; // End the simulation
end

endmodule
```

Attach screenshot of Simulation output- make sure to scale properly for visibility of all case.



c. Adder/Subtractor system

Provide code for design module here

```
`timescale 1ns / 1ps

module ADDERSUBTEST();

    // Inputs (Registers)
    reg [2:0] A;
    reg [2:0] B;
    reg O;

    // Outputs (Wires)
    wire [6:0] seg;
    wire dp;
    wire [3:0] an;

    // Instantiate the Unit Under Test (UUT)
    // Make sure 'ADDERSUB' matches the exact name of your top module
    ADDERSUB uut (
        .A(A),
        .B(B),
        .O(O),
        .seg(seg),
        .dp(dp),
        .an(an)
    );
endmodule
```

```
// Apply Test Stimulus
initial begin
    // Case 1: 3 + 4 [cite: 59]
    A = 3'b011; // 3 in binary
    B = 3'b100; // 4 in binary
    O = 1'b0; // 0 for Addition
    #10; // Wait 10 time units

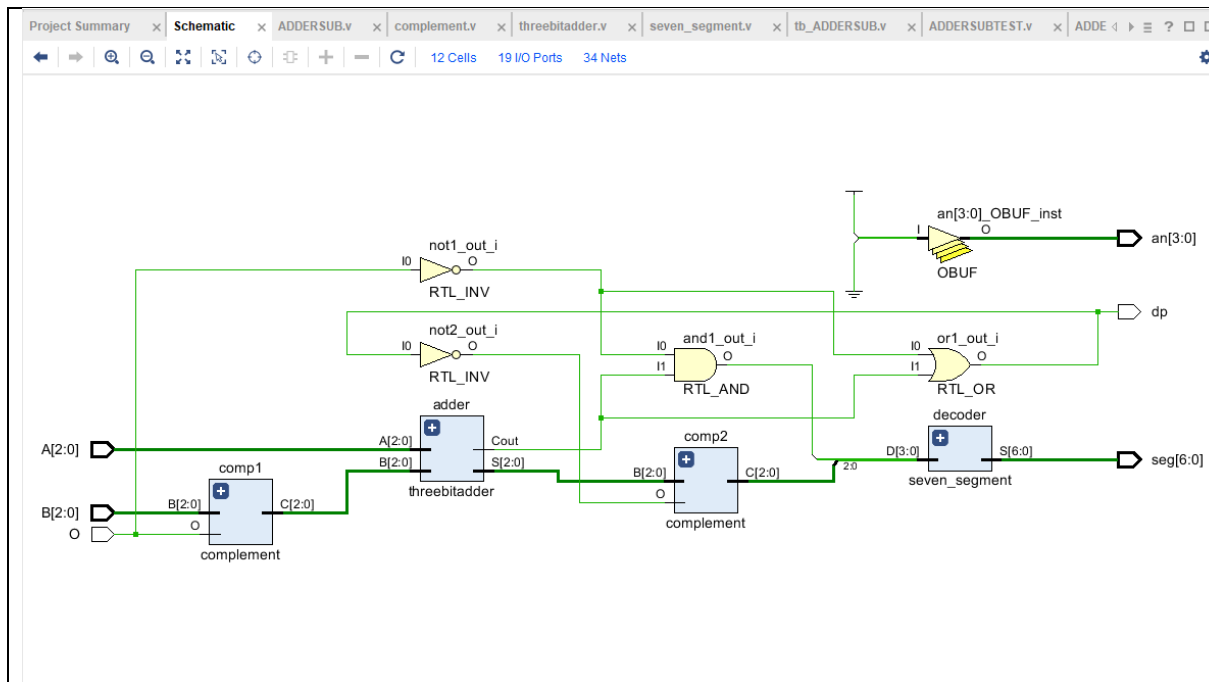
    // Case 2: 3 - 4 [cite: 60]
    A = 3'b011;
    B = 3'b100;
    O = 1'b1; // 1 for Subtraction
    #10;

    // Case 3: 7 + 7 [cite: 61]
    A = 3'b111; // 7 in binary
    B = 3'b111; // 7 in binary
    O = 1'b0;
    #10;

    // Case 4: 2 - 6 [cite: 62]
    A = 3'b010; // 2 in binary
    B = 3'b110; // 6 in binary
    O = 1'b1;
    #10;

    // End simulation
    $finish;
end
endmodule
```

Provide RTL Schematic here



Add your testbench here

```
`timescale 1ns / 1ps

module tb_ADDERSUB();

    // Inputs (Registers)
    reg [2:0] A;
    reg [2:0] B;
    reg O;

    // Outputs (Wires)
    wire [6:0] seg;
    wire dp;
    wire [3:0] an;

    // Instantiate the Unit Under Test (UUT)
    ADDERSUB uut (
        .A(A),
        .B(B),
        .O(O),
        .seg(seg),
        .dp(dp),
        .an(an)
    );

    // Apply Test Stimulus
    initial begin
        // Case 1: 3 + 4
        A = 3'b011; // 3 in binary
```

```
B = 3'b100; // 4 in binary
O = 1'b0; // 0 for Addition
#10; // Wait 10 time units

// Case 2: 3 - 4
A = 3'b011;
B = 3'b100;
O = 1'b1; // 1 for Subtraction
#10;

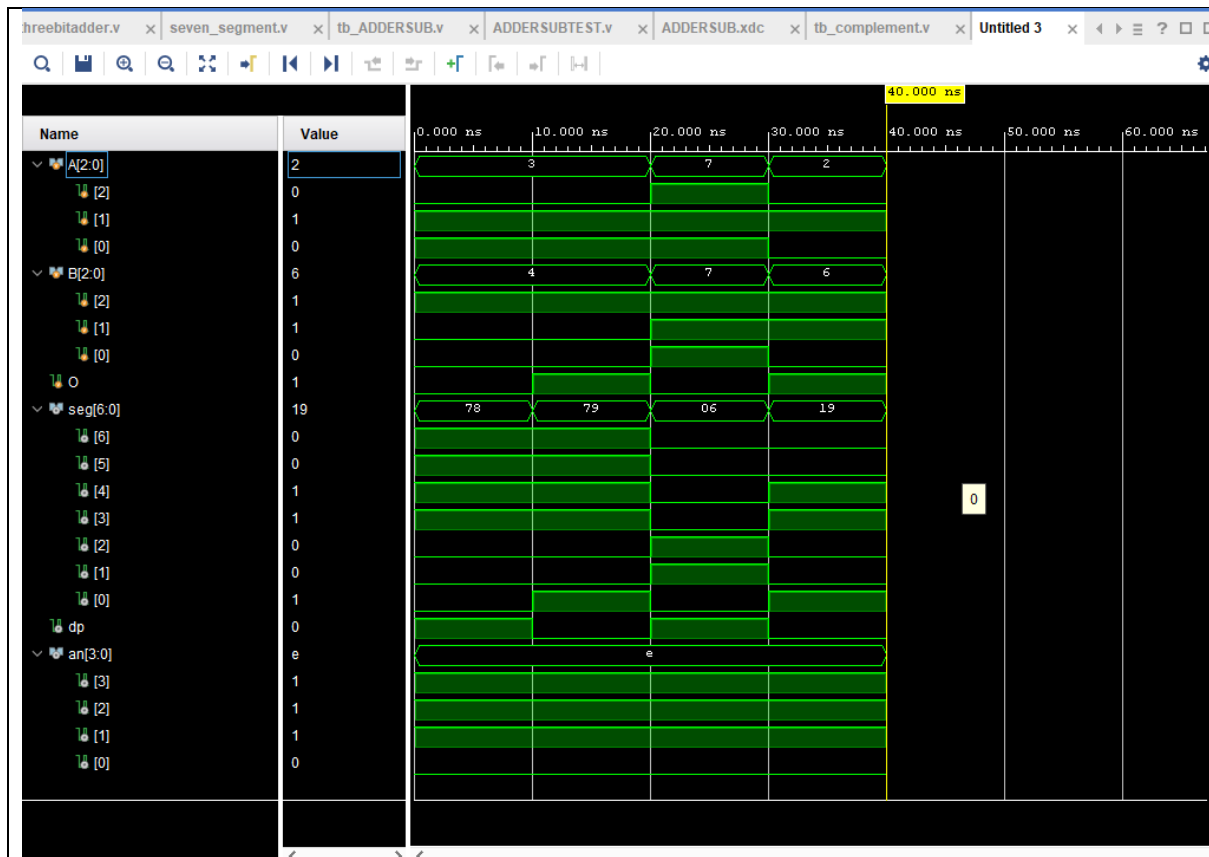
// Case 3: 7 + 7
A = 3'b111; // 7 in binary
B = 3'b111; // 7 in binary
O = 1'b0;
#10;

// Case 4: 2 - 6
A = 3'b010; // 2 in binary
B = 3'b110; // 6 in binary
O = 1'b1;
#10;

// End simulation
$finish;
end

endmodule
```

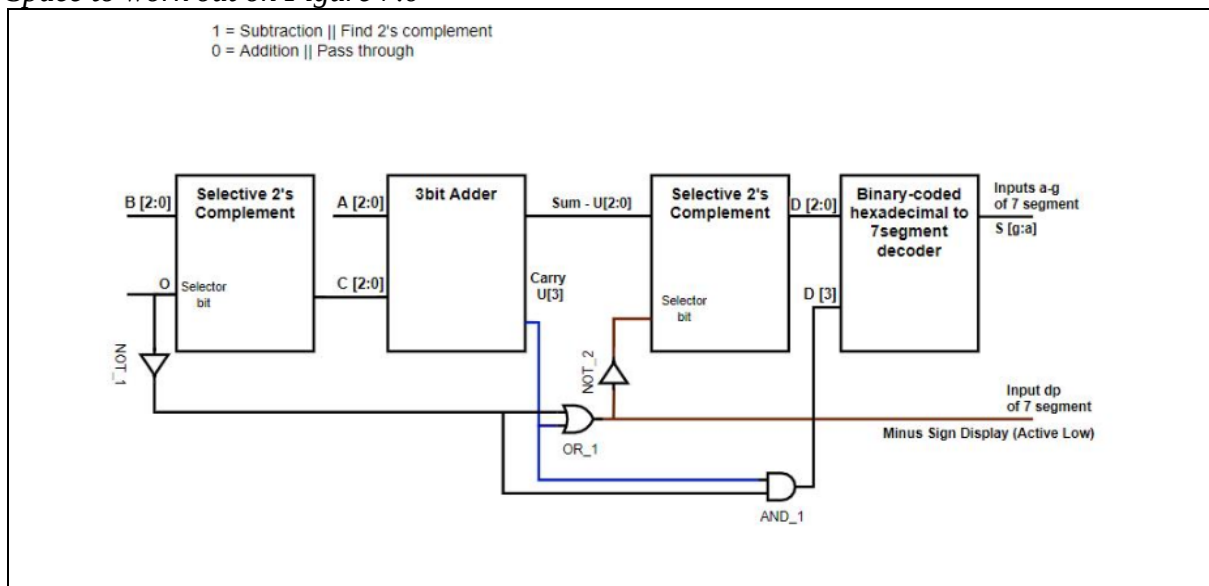
Attach screenshot of waveform results here



a. Exercise

Use Figure 6.0 and work out for cases when B is zero and subtracted from A. Reflect on the output obtained and suggest ways to fix (if required).

Space to work out on Figure 7.0



Space for reflection

Subtracting zero erroneously triggers the negative sign because the operation fails to generate a carry-out. To fix this, I updated the dp logic to include a condition for B=0. This modification forces the indicator to stay off whenever the subtrahend is zero, ensuring accurate positive results.

Lab Evaluation Rubrics

Marks Distribution:

		LR2 Code	LR4 Data Collection	LR5 Results	LR7 Viva	LR10 Analysis
In-lab	Task b	20 points	5points	10 points	10 points	-
	Task c	20 points	-	20 points		-
Exercise	Task d	-	-	-		15 Points
Total Marks =	100	40	5	30	10	15

Marks Obtained:

		LR2 Code	LR4 Data Collection	LR5 Results	LR7 Viva	LR10 Analysis
In-lab	Task b					-
	Task c		-			-
Exercise	Task d	-	-	-		
Total Marks =	100					